Fall 2026

CS-UY 2413 (Sections A and B): Design and Analysis of Algorithms

Prof. Lisa Hellerstein
lisa.hellerstein@nyu.edu

Slides include material from Introduction to Algorithms, 4th Edition, MIT Press by Cormen, Leiserson, Rivest, and Stein.

Lecture 9

- Recommended reading:
  - Chap. 6 (Heaps, Heapsort, Priority Queues)
  - Probability Handout on EdStem
  - Appendix C (Counting and Probability)
- Reading ahead
  - Chap. 5 Randomized Algorithms
  - Chap. 7 Quicksort

Some important concepts from probability

Linearity of Expectation

- We will use following important property called “Linearity of Expectation”
- For any two random variables X and Y, and constants a and b
  - $E[aX + bY] = aE[X] + bE[Y]$
- True even if X and Y are not independent!

A coin flipping problem

- Suppose flip a fair coin repeatedly, stopping as soon as get Heads, or have flipped 4 times (get 4 Tails)
- What is expected number of flips?
- Standard way to solve this:
  - Define X to be random variable which is number of flips performed (stop when get Heads)
  - $E[X]$ = Expected number of flips

$$= 1 * P[X = 1] + 2 * P[X = 2] + 3 * P[X = 3] + 4 * P[X = 4]$$

$$= 1 * \frac{1}{2} + 2 * \frac{1}{4} + 3 * \frac{1}{8} + 4 * \frac{1}{8}$$

A coin flipping problem

- Suppose flip a fair coin repeatedly, stopping as soon as get Heads, or have flipped 4 times (get 4 Tails in a row)
- What is expected number of flips?
- Standard way to solve this:
  - Define X to be random variable which is total number of flips performed (if stop when get Heads or flipped 4 Tails in a row)
  - $E[X]$ = Expected number of flips

$$= 1 * P[X = 1] + 2 * P[X = 2] + 3 * P[X = 3] + 4 * P[X = 4]$$

$$= 1 * \frac{1}{2} + 2 * \frac{1}{4} + 3 * \frac{1}{8} + 4 * \frac{1}{8}$$

$$= (1/2) + (1/2) + (3/8) + (4/8) = 1\frac{7}{8}$$

A different approach to coin flipping problem

- Can solve using Indicator Random Variables
- Indicator random variable Y for an event
  - $Y = 1$ if event occurs
  - $Y = 0$ if event does not occur
- Since Y is an indicator random variable, $E[Y] = P[Y = 1]$
  - Why? $E[Y] = 0 * P[Y=0] + 1 * P[Y=1]$

Solving coin flipping problem with indicator variables

- Again, suppose you have a fair coin
- Flip it repeatedly until either you get Heads or you've flipped it 4 times
- What is the expected number of flips you perform?
  - Let X_i be indicator variable for the event that you perform an ith flip
  - Note that the different X_i variables are not independent!

e.g. T T H $X_1=1$ $X_2=1$ $X_3=1$ $X_4=0$

- Let X be random variable which is total number of flips performed
- $X = X_1 + X_2 + X_3 + X_4$
- Expected number of flips performed is
  - $E[X] = E[X_1 + X_2 + X_3 + X_4] = E[X_1] + E[X_2] + E[X_3] + E[X_4]$

- Let X be random variable which is total number of flips performed
- $X = X_1 + X_2 + X_3 + X_4$
- Expected number of flips performed is
  - $E[X] = E[X_1 + X_2 + X_3 + X_4]$
    $= E[X_1] + E[X_2] + E[X_3] + E[X_4]$
    $= 1 + (1/2) + (1/4) + (1/8)$
    $= 1\frac{7}{8}$

$= P[X_1 = 1] + P[X_2 = 1] + P[X_3 = 1] + P[X_4 = 1]$

$P[\text{first flip heads}]$

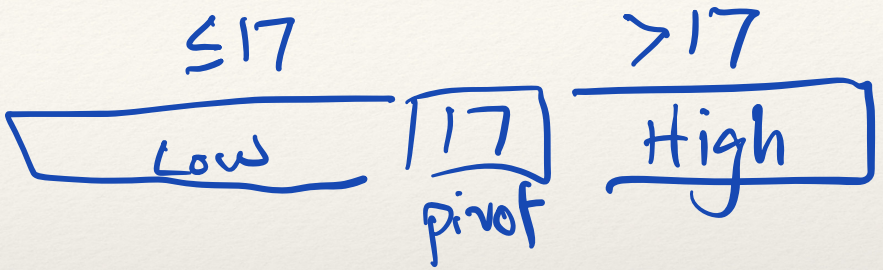
$P[\text{first 2 flips heads}]$

- Let X be random variable which is total number of flips performed
- $X = X_1 + X_2 + X_3 + X_4$
- Expected number of flips performed is
  - $$\begin{aligned} E[X] &= E[X_1 + X_2 + X_3 + X_4] \\ &= E[X_1] + E[X_2] + E[X_3] + E[X_4] \\ &= 1 + (1/2) + (1/4) + (1/8) \\ &= 1\frac{7}{8} \end{aligned}$$

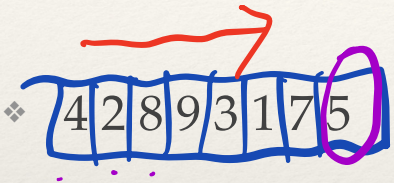
Quicksort

Quicksort

- Very practical (with tuning)
- Divide-and-conquer
- Idea:
  - Pick an element x of the array. Call it the "pivot"
  - Partition the other elements with respect to the pivot, as follows:
    Elements ≤ x go on left side of array (low side), elements > x go on right side of array (high side)
    Pivot x goes in between the two sides
  - Recursively sort the low side
  - Recursively sort the high side



Partition example





- Suppose pick 5 to be the pivot
- If use extra array, very easy to partition around pivot in time $\theta(n)$
- Can be done in-place in time $\theta(n)$ (slightly more complicated)

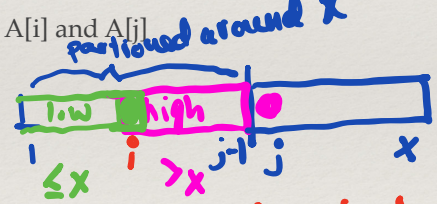
Partition example

- Idea for partitioning A[1:n] in place (pseudocode on p. 184, for A[p…r])
  - Pick A[n] to be pivot x
  - Use loop to partition longer and longer prefixes of A around pivot x
  - for j = 1 to n-1

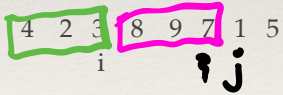
    \\ Loop invariant: Elements in A[1:j-1] partitioned into "low side" (elements <= x) and "high side" (elements > x)
    A[1:i] is low side, ~~A[i+1:j-1]~~ is high side

    A[i+1:j-1]

    - if A[j] > x, leave A[j] where it is, don't update i
    - if A[j] <= x, increment i then exchange A[i] and A[j]
  - At end, exchange A[i+1] and A[n]
  - (Need to initialize i to 0 at start)



- e.g. 4 2 3 8 9 7 1 5 fast forward to j=6



i is last position of low.

g. 184

```
PARTITION(A, p, r)
1  x = A[r]                              // the pivot
2  i = p − 1                             // highest index into the low side
3  for j = p to r − 1                    // process each element other than the pivot
4      if A[j] ≤ x                       // does this element belong on the low side?
5          i = i + 1                         // index of a new slot in the low side
6          exchange A[i] with A[j]           // put this element there
7  exchange A[i + 1] with A[r]           // pivot goes just to the right of the low side
8  return i + 1                          // new index of the pivot
```

- Note:
  - In mergesort — divide is trivial, combining results of recursive calls takes $\theta(n)$
  - In quicksort — divide takes work (partition), combining results of recursive calls is trivial

```
QUICKSORT(A, p, r)
1  if p < r
2      // Partition the subarray around the pivot, which ends up in A[q].
3      q = PARTITION(A, p, r)
4      QUICKSORT(A, p, q − 1)  // recursively sort the low side
5      QUICKSORT(A, q + 1, r)  // recursively sort the high side
```

Sorting A[p:r]

Running Time?

```
QUICKSORT(A, p, r)
1  if p < r
2      // Partition the subarray around the pivot, which ends up in A[q].
3      q = PARTITION(A, p, r)
4      QUICKSORT(A, p, q − 1)  // recursively sort the low side
5      QUICKSORT(A, q + 1, r)  // recursively sort the high side
```

Running Time? Varies depending on how the pivots split the array

Suppose pivot always split array in half

- If pivot always splits array exactly in half:

$$T(n) = 2T(n/2) + \theta(n)$$

- Running time?

$\Theta(n \log n)$

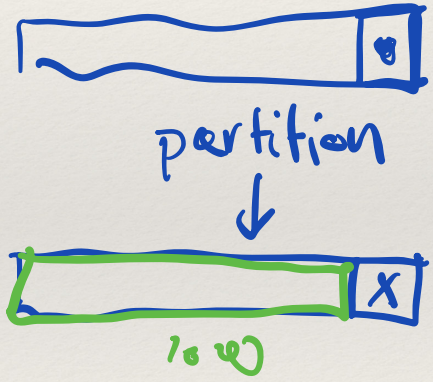
Suppose pivot is always max

- If pivot is always the maximum (or minimum) elements in the subarray, then

$$T(n) = T(n-1) + T(0) + \theta(n)$$

- equivalently

- $$T(n) = T(n-1) + \theta(n)$$



- Solve this recurrence

$T(n) = T(n-1) + cn$
$= T(n-2) + c(n-1) + cn$
$= T(n-3) + c(n-2) + c(n-1) + cn$

$$= T(n-3) + c\left((n-2) + (n-1) + n\right)$$

$$\vdots$$

$$= T(n-k) + c\left((n-k+1) + (n-k) + \cdots + n\right)$$

$$\vdots$$

$$= T(1) + c(2 + 3 + 4 + \cdots + n)$$

$$= \Theta(n^2)$$

Suppose pivot is always max

- If pivot is always the maximum (or minimum) elements in the subarray, then

$$T(n) = \theta(n^2)$$

- Is this actually the worst case??
- Yes!

Fact: Worst-case runtime of QuickSort is $O(n^2)$ (upper bound)

Why?

~~Suppose pivot is always max~~

- Lemma: Once Quicksort compares two elements of the input array, it will never do that comparison again.
  - Pf: Suppose elements are z_i and z_j. Consider first time they are compared. It must be during a call to Partition, so one of them is the pivot in that call. After that call, the pivot used in that call is never involved in any further comparisons. So z_i and z_j never compared to each other again.
- From this Lemma, it follows that

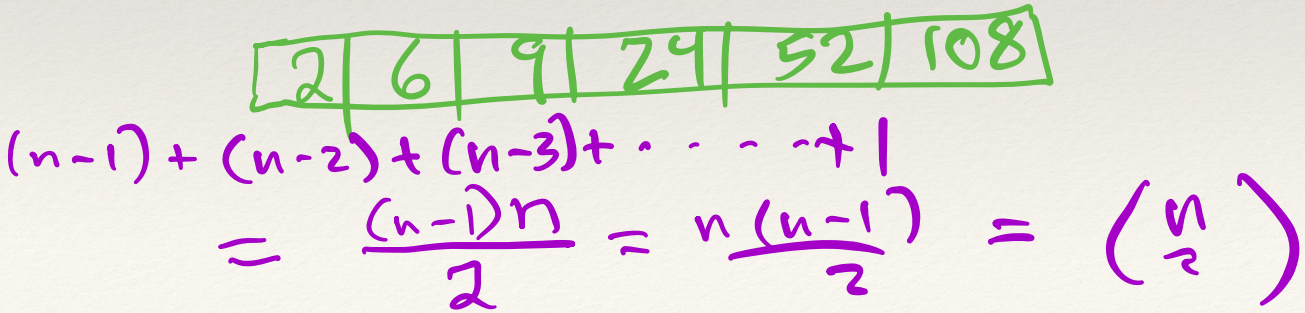
  # comparisons of pairs of array elements performed by Quicksort ≤ total number of pairs A of array elements
- = ???? $\binom{n}{2} = \frac{n(n-1)}{2}$
- Quicksort runtime (worst-case) is O(total number of pairs of array elements)

$O(n^2)$

Quicksort on already sorted array

- How many comparisons between array elements are performed by Quicksort if input array A is already sorted?

  (assuming pivot is chosen to be last element of subarray)



Quicksort on already sorted array

- How many comparisons between array elements are performed by Quicksort if input array A is already sorted?

  (assuming pivot is chosen to be last element of subarray)

  $n(n-1)/2$

- Already sorted array is a worst-case input for Quicksort (assuming pivot chosen to be last element of subarray)
  - contrast to e.g., Insertion sort

Quicksort runtime

- Runtime of Quicksort (worst case) is $\theta(n^2)$

Avoiding bad splits

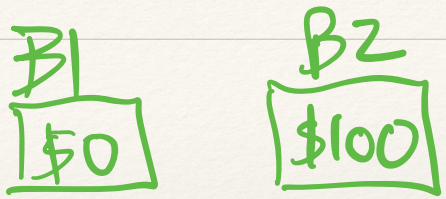
- Idea:
  - Choose a random element to be the pivot!
- This results in a Randomized Algorithm
  - Algorithm uses randomness in its execution (in practice, random number generator)
  - Execution of the algorithm on an input x is not always the same, depends on the random choices made by the algorithm

Runtime of randomized algorithms?

- Instead of asking:
  - What is the runtime of Algorithm Q on input x
- Ask:
  - What is the expected runtime of Algorithm Q on input x
- We talk about the "worst-case running time" of a deterministic (not randomized) algorithm A
  - Means maximum running time of Algorithm Q on any input x
- When Algorithm Q is randomized, we will talk about the "worst-case expected running time" of A
  - Means maximum "expected running time" of Algorithm Q on any input x
  - "worst-case" still refers to worst-case of input x, doesn't refer to being very unlucky with randomness

Analogy

Analogy



- Suppose I have 2 boxes, Box 1 and Box 2
- I put $100 in one box, and $0 in the other (you can't see which)
- You get to open one box and take what is inside
- You use the following deterministic (non-randomized) algorithm D for choosing the box:
  - Choose Box 1
- I can put the $100 in Box 2 (this is worst-case for you). You will get $0
- What if you use the following randomized algorithm R for choosing the box:
  - Flip a fair coin to decide which box to choose.
- No matter where I put the $100, the expected amount of money you get is

$50

Back to analyzing Randomized Quicksort

Randomized Quicksort

- (Deterministic) Quicksort always performs n(n-1)/2 comparisons if input array was already sorted.
  - Quicksort is doomed to perform badly if input array is sorted
- Randomized-Quicksort not doomed in this way
  - No matter what input array is given, *expected* number of comparisons performed by Randomized Quicksort will be much less than $n(n-1)/2$ (as long as n not very small).

- What is expected runtime of Randomized Quicksort?

Intuition

- What if pivot always divides n elements of subarray into $\frac{1}{10}n$ and $\frac{9}{10}n$

  $T(n) = T(n/10) + T(9n/10) + \theta(n)$

  - Runtime is $\theta(n \log n)$, same as $\frac{1}{2}n$ and $\frac{1}{2}n$ split every time
    - can show with recursion tree

Analysis of runtime of quick sort

- We will prove that expected runtime of Randomized Quicksort is $\theta(n \log n)$
- Will do probabilistic analysis
  - Review your probability
    - Handout on EdStem
    - Appendix C
    - (Also, Chap. 5 on Randomized Algorithms)
-

Analysis of runtime of quick sort

- To simplify the analysis, we assume all elements in the array are distinct
  - Analysis can be modified to deal with repeated elements, bound doesn't change

$$\sum_{i=1}^{n} \frac{1}{i}$$

- Harmonic series

$$\sum_{i=1}^{\infty} \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \frac{1}{5} + \cdots$$

- Does this series converge?

No

- Harmonic series

$$\sum_{i=1}^{\infty} \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \ldots$$

- Does this series converge? No.

- $\sum_{i=1}^{n} \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \ldots + \frac{1}{n}$ is called the n-th Harmonic number, denoted by H_n
- Theta bound on nth Harmonic number?

$$\sum_{i=1}^{n} \frac{1}{i} = \theta(\qquad\qquad)$$

$H_n =$

- Fact: $\sum_{i=1}^{n} \frac{1}{i} = \theta(\log n)$
  - Proof:
    - Use integral approximations to show

$$\sum_{i=1}^{n} \frac{1}{i} \leq \int_{x=1}^{n} \frac{1}{x} dx + 1 = \ln n + 1$$

$$\sum_{i=1}^{n+1} \frac{1}{i} \geq \int_{i=1}^{n+1} \frac{1}{x} dx = \ln(n+1)$$

    - (Appendix A, p. 1150 of textbook)

